Lecture 10: File Systems; Protection

Introduction to Computer Systems

Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Spring 2026

Contents

1 Introduction **3**

2 From Storage Blocks to File-System Abstractions **3**
2.1 The hardware interface: sectors, blocks, and logical block addresses . . . 3
2.2 The user's view and the operating system's view . . . 4
2.3 Files are named durable objects . . . 4
2.4 The file as the smallest logical storage unit . . . 5
2.5 Main components of a file system . . . 5

3 File Metadata and File Operations **6**
3.1 File descriptors, inodes, and file control blocks . . . 6
3.2 Opening a file . . . 6
3.3 Why user programs receive file handles instead of inode pointers . . . 7
3.4 Two levels of open-file tables . . . 7
3.5 Basic file operations . . . 8
3.6 File access patterns . . . 9
3.6.1 Sequential access . . . 9
3.6.2 Random access . . . 9
3.6.3 Keyed or indexed access . . . 9

4 Data Block Organization **10**
4.1 Design constraints . . . 10
4.2 Contiguous allocation . . . 10
4.3 Linked allocation . . . 11
4.4 The Windows FAT idea . . . 11
4.5 Indexed allocation . . . 12
4.6 Multi-level indexed allocation in 4.3 BSD Unix . . . 12
4.7 Modern extensions: extents and delayed allocation . . . 14

5 Free Space Management **14**
5.1 Linked free lists . . . 14
5.2 Bitmaps . . . 15

6 Directories, Naming, and Links **15**
6.1 Directories as name-to-number maps . . . 15
6.2 Unix/Linux directory structure . . . 16
6.3 Resolving a path . . . 16
6.4 Current working directory . . . 17
6.5 Hard links . . . 17
6.6 Symbolic links . . . 18

7 Buffer Cache and File-System Performance **18**
7.1 Why cache disk blocks? . . . 18
7.2 Read path with a buffer cache . . . 19
7.3 Buffer cache and virtual memory . . . 19
7.4 Synchronous writes and delayed writes . . . 19
7.4.1 Synchronous writes . . . 20
7.4.2 Delayed writes . . . 20
7.5 Performance pitfalls in user programs . . . 20

8 Protection **21**
8.1 Protection goals . . . 21
8.2 Authentication with passwords . . . 21
8.3 Attaching principals to processes . . . 22
8.4 Authorization as an access matrix . . . 22
8.5 Access-control lists . . . 22
8.6 Capabilities . . . 23
8.7 Access enforcement and the trusted computing base . . . 24

9 Worked Examples and Design Comparisons **24**
9.1 Counting block accesses in path lookup . . . 24
9.2 Maximum file size with multi-level indexing . . . 24
9.3 Allocation strategy comparison . . . 24

10 Summary **25**

11 Quizzes & Exercises **26**
11.1 Conceptual quiz . . . 26
11.2 Answer key . . . 26
11.3 Essay questions . . . 27
11.4 Additional practice problems . . . 27
11.5 Quick reference glossary . . . 28

1 Introduction

Persistent storage is one of the most important abstractions in a computer system. A program can allocate memory, manipulate pointers, and build rich in-memory data structures, but ordinary memory does not survive process termination or power loss. A file system provides the durable counterpart: it gives users and programs named objects whose contents can outlive the processes that created them, and it organizes those objects into a structure that can be searched, shared, protected, and recovered.

The file-system abstraction hides a large gap between what users want and what storage devices provide. A user wants to write a source file named `example.cpp`, search for it later by name, append a few bytes, or copy it to another computer. A disk or SSD provides a much lower-level interface: read or write a block, addressed by a logical block number. The operating system bridges these views. It maps names to metadata, metadata to disk blocks, and disk blocks to bytes that user programs can read and write.

This lecture also connects file systems with protection. Once files are durable and named, they become independent of the process that created them. That is useful because another process, another user, or another machine can access the same object. It is also dangerous: the system must prevent accidental corruption and malicious misuse. Therefore the same operating system that maps files onto blocks must also decide who is allowed to read, write, execute, rename, or delete them.

Definition

A **file system** is an operating-system component that transforms a block-oriented storage interface into a higher-level interface consisting of named files, directories, metadata, and protection rules.

The material in this lecture is organized around four persistent questions:

1. **Representation:** What metadata describes a file, and how is that metadata represented on disk and in memory?
2. **Allocation:** Which disk blocks store the file contents, and how does the system find free blocks for new data?
3. **Naming:** How does a path such as `/my/book/count` become a particular file descriptor and then a set of disk blocks?
4. **Protection:** How does the system decide whether a process may perform a requested operation on a protected object?

2 From Storage Blocks to File-System Abstractions

2.1 The hardware interface: sectors, blocks, and logical block addresses

Secondary storage devices are normally accessed in units larger than a single byte. A hard disk stores data in physical sectors, while an operating system usually issues transfers in larger logical blocks. Unix-like systems often use 4 KiB blocks, although real systems may use several block sizes for different layers. SSDs expose a block-like interface to the operating system but internally organize flash memory into pages and erase blocks.

Definition

A **sector** is a physical storage unit on a device. A **block** is a logical data-transfer unit used by the operating system or file system. One block may consist of one or more sectors.

Modern systems usually address storage using **Logical Block Addresses** (LBAs). LBA 0 is followed by LBA 1, and so on, up to the maximum number of logical blocks. This address space is linear even when the physical medium is not. A disk controller can skip bad sectors, and an SSD controller can perform wear-leveling by remapping frequently written logical blocks to different physical flash pages. The operating system generally does not need to know the changing physical position of each LBA.

This hardware abstraction is powerful because it separates concerns. The storage controller hides device-specific details such as bad-sector remapping, flash translation layers, and wear-leveling. The operating system can then build a file system over a simple model: a long numbered array of blocks.

2.2 The user's view and the operating system's view

Users see files as durable data structures. A text file, object file, database file, image, or executable is identified by a name and can be opened later. The system-call interface mostly treats a file as a sequence of bytes. A program can ask to read the next 100 bytes, write bytes 100 through 200, seek to an offset, or synchronize changes to persistent storage. The file system itself is usually oblivious to the meaning of those bytes. It does not need to know whether the bytes encode C source code, a JPEG image, an SQLite database, or compressed data.

Internally, the operating system views the same file as a collection of disk blocks. Those blocks may be contiguous, scattered across the disk, chained through a table, or reachable through an index structure. A byte-level operation is therefore implemented using block-level operations. If the program writes one byte in the middle of a block, the operating system must identify the containing block, bring the block into memory if necessary, modify the byte in memory, and eventually write the whole block back to storage.

Deep Dive

A storage device normally cannot update an arbitrary byte in isolation. Even a one-byte logical write becomes a read-modify-write at block granularity unless the block is already cached. This difference between byte-level program interfaces and block-level device interfaces explains why file systems rely heavily on buffer caches and why small random writes can be expensive.

The number of allocated blocks for a file of size S bytes and block size B bytes is

$$N_{blocks} = \left\lceil \frac{S}{B} \right\rceil .$$

For example, if $S = 15000$ bytes and $B = 4096$ bytes, then $N_{blocks} = 4$. The last block contains only $15000 - 3 \cdot 4096 = 2712$ useful bytes, leaving $4096 - 2712 = 1384$ unused bytes. This unused space is internal fragmentation.

2.3 Files are named durable objects

Naming is central to the file abstraction. In memory, data structures are usually found by addresses; on disk, users and programs find durable objects by names such as `readme.txt`, `notepad.exe`, or `src/main.cpp`. Names are easier for humans to recognize than numeric addresses, and they also make files independent of the creating process. Once a file is named,

another process can open it by the same name, another user can receive it, and another system can store it while preserving the name.

A name is not merely cosmetic. It is the handle by which a durable object participates in sharing, backup, transfer, and protection. A compiler may create an object file, a linker may later read it, and an editor may display it. These programs do not need to know the physical blocks used by the file; they only need a path name and the appropriate permissions.

2.4 The file as the smallest logical storage unit

From a user's perspective, ordinary persistent data must live inside a file. Raw disk blocks exist, but normal user programs are not expected to allocate arbitrary disk addresses directly. This rule helps the operating system manage space, enforce permissions, maintain metadata, and provide recovery mechanisms.

Unix-like systems extend the idea even further with virtual file systems such as `/proc`. Many entries under `/proc` are not ordinary files stored on disk. Instead, reading from them asks the kernel to produce system information, and writing to some entries changes system configuration. Examples include `/proc/cpuinfo`, process-specific directories such as `/proc/567/`, and configuration trees such as `/proc/sys/`.

```
with open("/proc/cpuinfo", "r", encoding="utf-8") as f:
    first_lines = [next(f).rstrip() for _ in range(5)]

for line in first_lines:
    print(line)
```

Listing 1: Reading a virtual file in /proc using ordinary file operations.

The code uses normal file operations, but the data is generated by the kernel. This design illustrates a Unix principle: when possible, expose system state through the same interface used for ordinary data.

2.5 Main components of a file system

A file system consists of two large classes of data structures:

- **File data and metadata.** Each file stores useful data in disk blocks and has metadata describing its size, ownership, permissions, timestamps, and block locations.
- **Directory structure.** Directories organize names and map those names to files.

From these structures arise several responsibilities.

1. **Disk management:** Track which blocks belong to each file and which blocks are free.
2. **Naming:** Resolve paths into file numbers and file metadata.
3. **Protection:** Isolate users and processes by enforcing access rules.
4. **Reliability and durability:** Keep persistent data consistent despite crashes or failures.

This lecture emphasizes the first three. Crash consistency and transactions are deeper topics that appear later in operating systems and database systems.

3 File Metadata and File Operations

3.1 File descriptors, inodes, and file control blocks

A file system must remember more than the bytes stored in a file. It must remember how many bytes the file has, which blocks store those bytes, who owns the file, what permissions apply, and when the file was last accessed or modified. This information is called file metadata.

Definition

A **file descriptor** in the file-system sense is an operating-system data structure containing a file's metadata. Unix-style systems call this structure an **inode**. Some operating-system texts call it a **file control block**.

The word "file descriptor" is unfortunately overloaded. In Unix programming, a file descriptor is also the small integer returned by `open`. In these notes, to avoid confusion, we use:

- **file number** or **i-number**: an index used to locate an inode on disk;
- **inode** or **file control block**: the metadata structure describing the file;
- **file handle**: the per-process integer returned to user space by `open`.

Typical inode metadata includes:

- file size;
- block-location information, such as direct and indirect block pointers;
- timestamps, such as creation time, last modification time, and last access time;
- owner ID and group ID;
- permission bits or richer access-control information;
- type bits, such as regular file, directory, symbolic link, device, or socket;
- link count, used to decide when an inode can be deleted.

Metadata must itself be persistent. If the data blocks survive a reboot but the system forgets which blocks belong to which file, the file is effectively lost. Therefore inodes are stored on disk. When a file is opened, relevant metadata is copied into kernel memory so that later operations can use it efficiently.

3.2 Opening a file

Opening a file resolves a human-readable name into kernel-managed state. A simplified `open` operation performs the following steps:

1. Search the directory structure using the path name.
2. Translate the path into a file number, such as a Unix i-number.
3. Use the file number to locate the inode on disk.
4. Copy or reference the inode in kernel memory.
5. Create open-file table entries.
6. Return a file handle, usually a small integer, to the user process.

Later calls such as `read`, `write`, `lseek`, `fsync`, and `close` operate on this file handle. The handle is meaningful only when interpreted by the kernel.

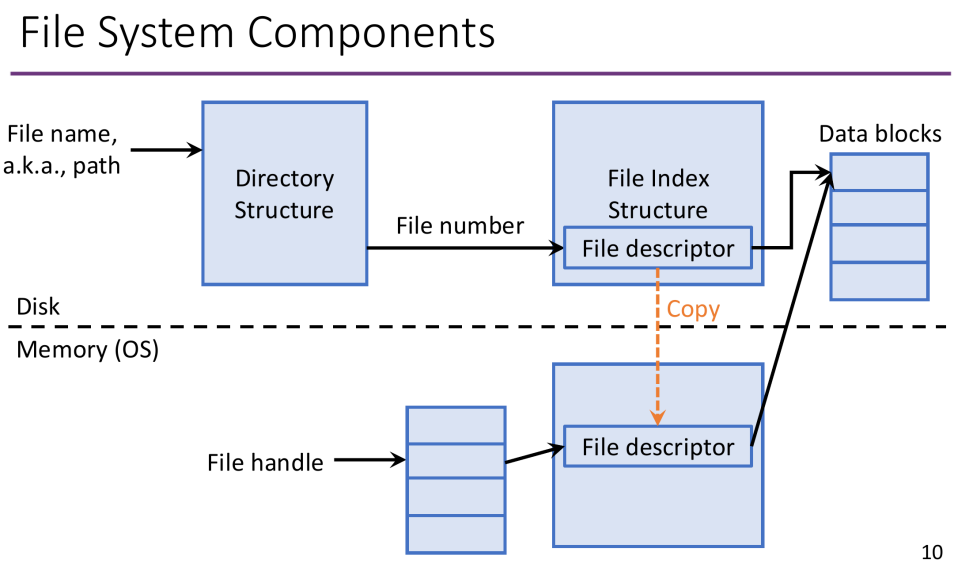


Figure 1: Path names are resolved through a directory structure into file numbers. File numbers locate persistent file descriptors or inodes on disk. When the file is open, the operating system keeps an in-memory representation and returns a file handle to the process. The handle is later translated by the kernel into open-file state and then into data block locations.

3.3 Why user programs receive file handles instead of inode pointers

A natural question is why `open` does not simply return a pointer to the in-memory inode. The reason is protection and abstraction. The inode lives in kernel memory. If a user process received a direct pointer to it, the process could attempt to read or corrupt kernel data structures, modify metadata, bypass permission checks, or damage other files. Even if the process is not malicious, a bug could overwrite important state.

A file handle is a controlled reference. It is a small integer in user space, but the actual table lookup and metadata access happen inside the kernel. The kernel can validate each operation, check permissions, maintain reference counts, and prevent direct modification of kernel structures. This pattern appears throughout systems: expose a handle, descriptor, or token to user code, but keep the authoritative object in a protected domain.

Deep Dive

A file handle is also a simple example of a capability-like reference. A process cannot normally invent a valid file handle for a file it has not opened, because the handle is interpreted relative to the process's open-file table. This is not a complete capability system, but it illustrates a recurring idea: give user code an unforgeable or hard-to-forge name for an object, and make the kernel mediate all operations on that name.

3.4 Two levels of open-file tables

A running system may have many processes, and multiple processes may open the same file simultaneously. The kernel therefore uses open-file state in more than one layer.

The **per-process open-file table** maps file handles to open-file entries. This table is process-local: file handle 3 in one process need not refer to the same file as file handle 3 in another process. This locality is useful because each process has its own view of its open resources.

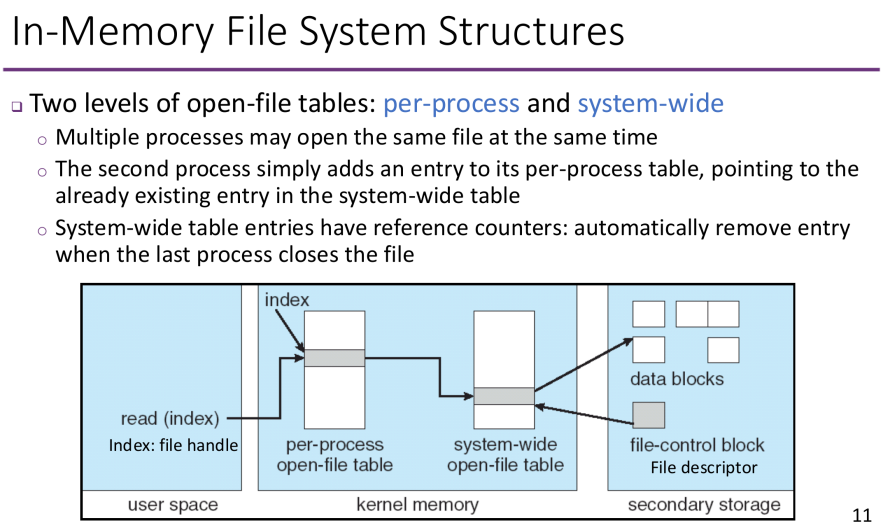


Figure 2: In-memory file-system structures often contain a per-process open-file table and a system-wide open-file table. A file handle indexes the per-process table. Multiple per-process entries may point to the same system-wide open-file object, which in turn references the inode and the data blocks. Reference counts allow the system-wide entry to be removed after the last process closes the file.

The **system-wide open-file table** stores shared information about opened files. If two processes open the same file, the second process can add an entry to its own per-process table that points to an already existing system-wide entry. The system-wide entry can hold or reference the in-memory inode, status flags, and other shared state. It also uses a reference count. When a process closes its handle, the corresponding per-process entry is removed; the shared entry is freed only after no process refers to it.

Real Unix kernels have additional distinctions, for example between an inode or vnode, an open file description, and a file descriptor table entry. The important conceptual point is that different state belongs at different sharing levels. Some state is per process; some state is shared by all processes that refer to the same open file; some state is persistent metadata for the file itself.

3.5 Basic file operations

The common system calls form a minimal interface to the byte-sequence abstraction:

- `open(path, flags, mode)` resolves a path and returns a handle.
- `read(fd, buffer, n)` copies up to n bytes from the file into memory.
- `write(fd, buffer, n)` copies bytes from memory into the file.
- `lseek(fd, offset, whence)` changes the current file offset.
- `fsync(fd)` requests that dirty file data and required metadata be forced to persistent storage.
- `close(fd)` releases the handle and decrements reference counts.

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
```

```c

int main(void) {
    int fd = open("notes.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    const char *msg = "file systems map byte streams to blocks\n";
    ssize_t n = write(fd, msg, strlen(msg));
    if (n < 0) {
        perror("write");
        close(fd);
        return 1;
    }

    if (fsync(fd) < 0) {    /* ask the OS to push data to storage */
        perror("fsync");
        close(fd);
        return 1;
    }

    close(fd);
    return 0;
}
```

Listing 2: A small C example using file handles and explicit synchronization.

The call to `fsync` is important for durability. A successful `write` often means that bytes have reached kernel memory, not necessarily that they have reached persistent storage.

3.6 File access patterns

File-system data structures are shaped by how files are accessed.

3.6.1 Sequential access

Sequential access processes data in order. Examples include scanning a log, compiling a source file, copying a file, or parsing input line by line. This is the most common pattern for many ordinary files. A good file system should make sequential access fast, ideally by placing consecutive logical file blocks near each other and by prefetching blocks before they are requested.

3.6.2 Random access

Random access uses an offset to directly address a byte range in the middle of a file. A database may jump to a page containing a particular record. A virtual-memory system may page data in from a swap file. A program may read a header near the beginning and an index near the end. Random access should be much faster than reading all preceding bytes. This requirement strongly disfavors pure linked-list allocation for large files.

The file-system **seek** operation should not be confused with a mechanical disk-head seek. The former changes a logical file offset; the latter moves a disk head to a physical track. A logical seek may eventually cause a physical seek, but the concepts live at different abstraction layers.

3.6.3 Keyed or indexed access

Keyed access searches for content rather than an offset. For example, a user may ask for all files containing the word “system.” Most operating systems do not provide this as a primitive

operation over arbitrary file contents. Instead, search tools, databases, and indexing services build additional data structures. MacOS Spotlight is a familiar example of system-level content indexing.

4 Data Block Organization

4.1 Design constraints

A file is a byte sequence, but storage is allocated and transferred in blocks. The file system must decide which blocks belong to a file and how to find them quickly. This problem resembles memory allocation, but the performance goals differ because storage devices are slower and because files are persistent, named, and often grow over time.

Several empirical facts guide the design:

- Most files are small, often only a few kilobytes or less. Therefore per-file metadata overhead must be low.
- Most disk space is consumed by relatively rare large files. Therefore large-file throughput and random access matter.
- File sizes grow unpredictably. A file system should support append and extension without excessive copying.
- Locality matters. On disks, nearby blocks reduce mechanical seek cost. On SSDs, locality affects controller behavior, caching, and metadata overhead even though there is no disk head.

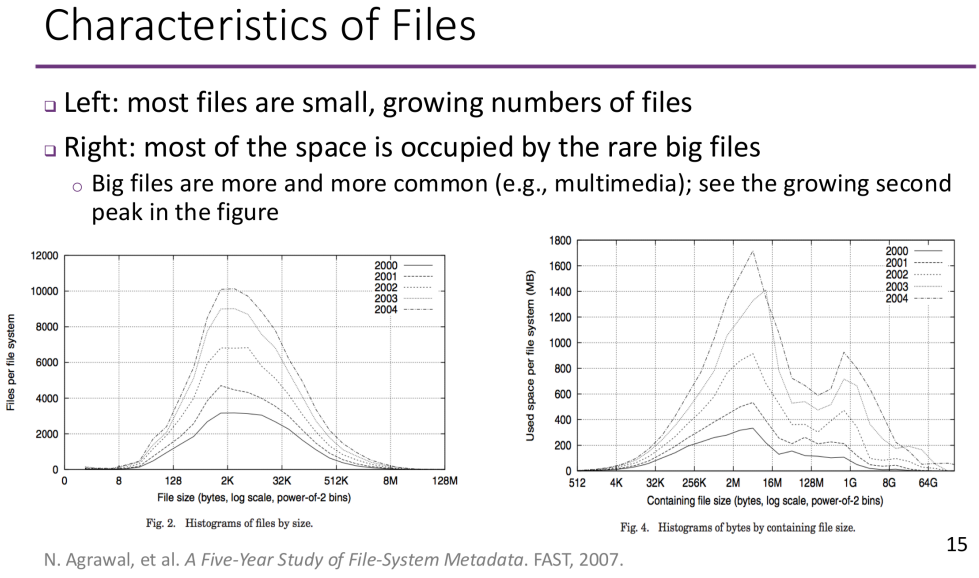


Figure 3: Empirical file-size distributions show two important facts. Counted by number, most files are small. Counted by occupied space, large files dominate. The second peak in the space-weighted distribution reflects the growing importance of multimedia and other large objects. A good file system must therefore serve both tiny files and large files efficiently.

4.2 Contiguous allocation

The simplest allocation strategy stores each file in a contiguous run of disk blocks. The inode needs only the starting block number and the number of blocks. If a file begins at block s, then logical block i of the file is stored at disk block $s + i$.

Definition

Contiguous allocation stores all blocks of a file in consecutive disk addresses. The file descriptor records the starting block and the length.

Contiguous allocation has attractive properties:

- It is simple to represent.
- Sequential access is fast because consecutive logical blocks are physically adjacent.
- Random access is fast because the target disk block can be computed by arithmetic.
- Metadata overhead is tiny.

However, contiguous allocation suffers from external fragmentation. As files are created and deleted, free space may become split into many small holes. The total free capacity may be large enough for a new file, but no single contiguous run may be large enough. File growth is also difficult. If another file begins immediately after the current one, appending more blocks may require moving the entire file to a larger free region.

The trade-off mirrors arrays in memory. Arrays support efficient indexing because elements are contiguous, but dynamically growing an array can require reallocation and copying when adjacent space is unavailable.

4.3 Linked allocation

Linked allocation removes the requirement that file blocks be contiguous. Each file is represented as a linked list of blocks. The inode stores a pointer to the first block; each block stores a pointer to the next block.

Definition

Linked allocation represents a file as a chain of blocks. Each block points to the next block in the file, and the inode points to the first block.

Linked allocation avoids external fragmentation. Any free block can be added to a file, and appending is straightforward. It can even support insertion in the middle of a file by changing pointers.

The cost is access performance. To find logical block i, the system must follow i links from the first block. Random access is therefore $O(i)$, which is unacceptable for large files. Even sequential access may have poor locality because consecutive logical blocks can be physically far apart. Each pointer also consumes space or requires a separate table.

4.4 The Windows FAT idea

The File Allocation Table (FAT) design keeps the linked-allocation idea but stores the links in a separate table rather than inside data blocks. Each FAT entry corresponds to one disk block. The entry stores the index of the next block in the file, or a special value meaning end-of-file or free block. The file descriptor stores the first block of the file.

During normal operation, the FAT is often kept in memory. This makes link traversal faster because the system can follow the chain in memory before issuing disk reads for data blocks. Older FAT variants used 16-bit entries, limiting the number of addressable blocks to 2^{16}. FAT32 uses larger entries; in practice, 28 bits are used for block addressing.

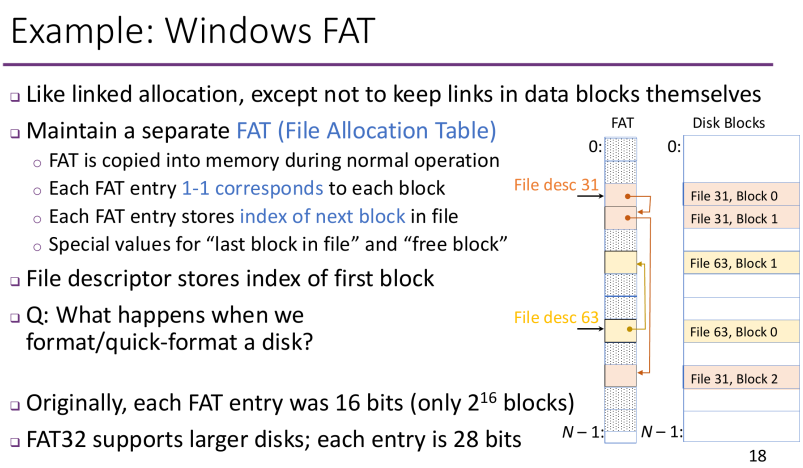


Figure 4: In FAT-like file systems, links are stored in a separate File Allocation Table. Each table entry corresponds to one disk block and stores the next block in the file. This preserves the flexibility of linked allocation while keeping data blocks free of embedded link fields.

Deep Dive

A quick format can make a FAT volume appear empty by clearing metadata such as the allocation table and root-directory information, without overwriting every data block. The old data may physically remain until reused. A full format or secure erase must overwrite or otherwise invalidate the data blocks themselves. This distinction explains why deleted or quick-formatted files can sometimes be recovered.

4.5 Indexed allocation

Indexed allocation stores pointers to a file's data blocks in an index block. The inode points to the index block; the index block contains an array of block pointers. Logical block i is found by reading pointer i in the index block.

Definition

Indexed allocation represents a file by storing the set of pointers to its data blocks in one or more index blocks. The file's data blocks need not be contiguous.

Indexed allocation combines several benefits of contiguous and linked allocation. It supports direct random access to block pointers and avoids external fragmentation because data blocks can be anywhere. It also keeps data blocks pure, with no embedded next pointers.

The main difficulty is choosing the size of the index. A small index limits maximum file size. A large index wastes space for small files. This is a classic systems design trade-off: a fixed-size structure is simple, but no single fixed size is ideal for both tiny and huge files.

4.6 Multi-level indexed allocation in 4.3 BSD Unix

The 4.3 BSD Unix design uses a multi-level index that adapts to file size. Assume 4 KiB blocks and 4-byte block pointers. Then one indirect block can store

$$\frac{4096}{4} = 1024$$

block pointers.

In the simplified version discussed here, the inode contains 14 block pointers:

- the first 12 point directly to data blocks;
- the next one points to a single-indirect block containing 1024 data-block pointers;
- the last one points to a double-indirect block containing pointers to indirect blocks, each of which contains data-block pointers.

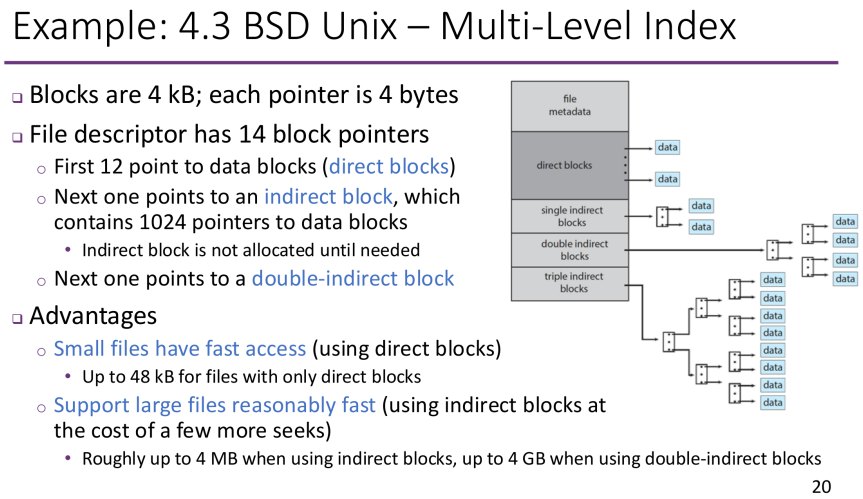


Figure 5: A multi-level indexed inode gives fast access to small files through direct pointers while supporting large files through indirect and double-indirect blocks. Indirect blocks are allocated only when needed, so small files avoid most indexing overhead.

The direct blocks cover

$$12 \times 4096 = 49152 \text{ bytes} = 48 \text{ KiB}.$$

The single-indirect block adds

$$1024 \times 4096 = 4 \text{ MiB}$$

of addressable data. The double-indirect block adds approximately

$$1024 \times 1024 \times 4096 = 4 \text{ GiB}.$$

Some Unix-derived file systems also use triple-indirect pointers or more modern extent-based designs, but the same principle remains: use cheap direct representation for small files and add levels of indirection only when the file becomes large.

Algorithm 1: Map a File Offset to a Disk Block in a BSD-Style Inode

Input: Inode I, byte offset x, block size $B = 4096$
Output: Disk block containing byte offset x

$j \leftarrow \lfloor x/B \rfloor$;
if $j < 12$ **then**
  return $I.direct[j]$;
$j \leftarrow j - 12$;
if $j < 1024$ **then**
  $S \leftarrow I.singleIndirect$;
  return $S[j]$;
$j \leftarrow j - 1024$;
$outer \leftarrow \lfloor j/1024 \rfloor$;
$inner \leftarrow j \bmod 1024$;
$D \leftarrow I.doubleIndirect$;
$S \leftarrow D[outer]$;
return $S[inner]$;

This algorithm demonstrates why multi-level indexing is effective. Direct access for small files requires only the inode. Single-indirect access requires one additional metadata block. Double-indirect access requires two. The overhead grows slowly while maximum file size grows rapidly.

4.7 Modern extensions: extents and delayed allocation

Many modern file systems generalize contiguous allocation using **extents**. An extent describes a contiguous run of disk blocks using a pair such as $(start, length)$. A large file can be represented by several extents instead of thousands of individual block pointers. Extents preserve locality when possible while avoiding the all-or-nothing limitation of requiring the whole file to be contiguous.

Modern systems also use delayed allocation. Instead of choosing disk blocks immediately when a program writes into the page cache, the file system may wait until dirty pages are flushed. By waiting, it can see larger write ranges and allocate more contiguous extents. This improves locality but complicates crash-consistency and free-space accounting.

5 Free Space Management

Data block organization answers the question: which blocks belong to this file? Free space management answers the complementary question: which blocks are currently available for allocation?

5.1 Linked free lists

Early systems often represented free blocks as a linked list. Each free block points to the next free block. Allocation removes a block from the list; deallocation inserts a block back into the list. This resembles free-list allocators for dynamic memory.

The linked-list approach is simple but not ideal for locality. It is also awkward when the system wants to find a free block near another block. Locality matters because placing newly appended blocks near the previous end of a file improves sequential performance.

5.2 Bitmaps

BSD Unix uses a bitmap. The file system keeps one bit per disk block. A bit value indicates whether the corresponding block is free or in use. The convention in the lecture is 1 for free and 0 for in use, although real systems may choose either convention.

Definition

A **free-space bitmap** is an array of bits, one per block, that records whether each block is free or allocated.

A bitmap makes it easy to search for a free block near a target position. When appending to a file, the system can start scanning near the file's previous last block. If the disk has plenty of free space, a nearby free block is usually found quickly, preserving locality.

Algorithm 2: Locality-Aware Free Block Search with a Bitmap

Input: Free bitmap M, preferred block p, total blocks N
Output: Allocated block number or failure

```
for d ← 0 to N − 1 do
    candidate ← (p + d) mod N;
    if M[candidate] = 1 then
        M[candidate] ← 0;
        return candidate;
return failure;
```

When the disk becomes nearly full, however, bitmap scanning can become slow and locality degrades. If only a few scattered free blocks remain, a search may scan many bits before finding one, and the chosen block may be far from the file's existing blocks.

The BSD approach reserves a fraction of capacity, traditionally around 10 percent. Once user-visible capacity is nearly full, the system can report the file system as full even though reserved blocks remain. This is a space-vs-time trade-off: sacrificing some capacity keeps allocation fast and helps preserve locality.

Deep Dive

The reserve is not merely an administrative convenience. It protects the allocator's performance model. A file system that is 99.9 percent full may still have free bytes, but they are likely to be fragmented and expensive to find. Reporting "full" earlier can be the correct systems decision when the alternative is severe slowdown and poor layout.

6 Directories, Naming, and Links

6.1 Directories as name-to-number maps

A directory maps names to file numbers. The file number then identifies the inode or file control block on disk. Early systems sometimes used a single directory for an entire disk. This was simple but caused name conflicts: if one user created `notes.txt`, no other user could use the same name on that disk. A later improvement was one directory per user, but that still made it difficult to organize large collections of files.

Modern systems use hierarchical directories. A directory can contain both files and other directories, producing a tree-like namespace. A path uses separators such as `/` to indicate levels. For example, `/usr/bin/python` begins at the root directory, then finds `usr`, then `bin`, then `python`.

6.2 Unix/Linux directory structure

Unix treats directories much like regular files. A directory has an inode, a type bit indicating that it is a directory, and data blocks. The data blocks contain directory entries. A simplified directory entry is a pair:

$$\langle \text{name}, \text{i-number} \rangle.$$

The directory entries form an unordered list or a more optimized structure in modern file systems. User programs can read directory contents through system calls such as `readdir`. Ordinary user programs usually cannot write raw directory data directly; instead, they request operations such as `mkdir`, `link`, `unlink`, and `rename`, and the kernel updates the directory safely.

In classic Unix conventions, the root directory has no ordinary name and is found at a fixed place. The lecture notes use i-number 2 for the root, i-number 0 as null, and i-number 1 for bad blocks.

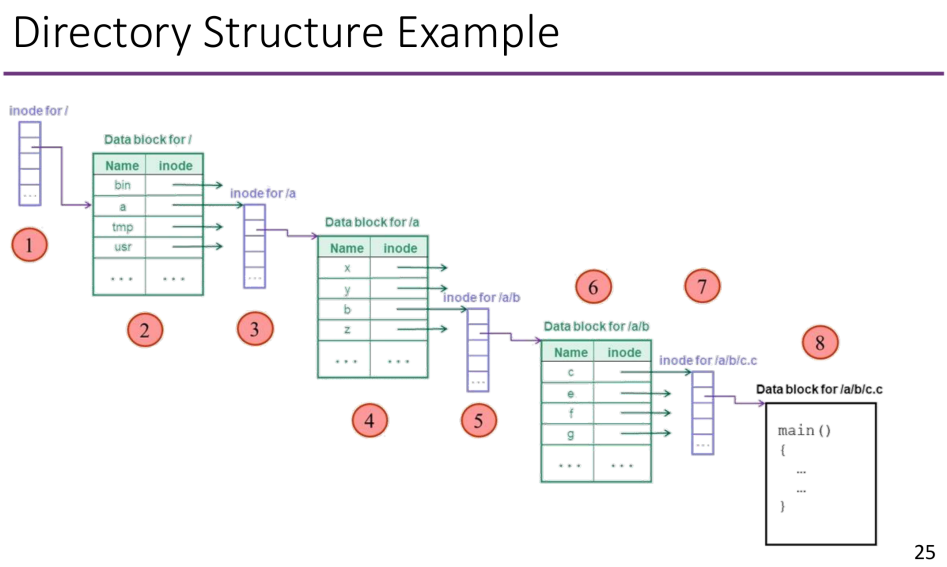


Figure 6: A hierarchical directory lookup alternates between reading an inode and reading the directory data block referenced by that inode. Each directory data block maps child names to child i-numbers, eventually leading to the target file's inode and then to its data blocks.

6.3 Resolving a path

Resolving `/my/book/count` illustrates the mechanism. A simplified lookup performs these disk accesses if nothing is cached and each directory fits in one data block:

1. Read the root inode at its fixed location.
2. Read the root directory's data block and search for `my`.
3. Read the inode for `/my`.
4. Read `/my`'s directory data block and search for `book`.
5. Read the inode for `/my/book`.
6. Read `/my/book`'s directory data block and search for `count`.
7. Read the inode for `/my/book/count`.

That is seven disk reads in this simplified model before reading any file data. If a directory spans multiple blocks, more reads may be required. If the buffer cache contains the root inode, directory blocks, or intermediate inodes, fewer physical storage accesses are required.

Algorithm 3: Path Resolution in a Hierarchical Directory Tree

Input: Path string P and process working directory *cwd*
Output: Inode of the named object, or failure

```
if P starts with "/" then
    node ← rootInode;
else
    node ← cwd;
Split P into components separated by "/";
foreach component c in order do
    if c is empty or c is "." then
        continue;
    if c is ".." then
        node ← parentDirectory(node);
        continue;
    if node is not a directory then
        return failure;
    Read directory entries of node;
    Find entry with name c;
    if no such entry exists then
        return failure;
    node ← inode(entry.inumber);
return node;
```

6.4 Current working directory

Using full path names all the time is cumbersome. The operating system therefore records a current working directory for each process, commonly in the process control block. If a path begins with `/`, lookup starts from the root. If it does not, lookup starts from the process's working directory.

The special entries `.` and `..` refer to the current directory and the parent directory. These entries make relative navigation possible and are central to command-line operations such as `cd ..`.

6.5 Hard links

A hard link creates another directory entry pointing to the same i-number as an existing file. The two path names are equal names for the same inode and therefore the same file data. The inode stores a link count recording how many directory entries point to it.

Definition

A **hard link** is a directory entry that maps a new name to an existing file number. It creates another path to the same inode.

Hard links make the directory structure a directed acyclic graph rather than a pure tree. In practice, Unix-like systems usually restrict hard links to directories, because hard-linking directories can create cycles and make traversal, deletion, and consistency checking much harder.

A file's data blocks can be deleted only when two conditions are satisfied:

1. the inode's link count is zero, meaning no directory entry names it;
2. no open-file table entry still references it.

This explains a common Unix behavior: if a process opens a file and another process unlinks it, the file name disappears from the directory, but the data remains available to the open process until it closes the file. The inode and blocks are freed only after the last reference is gone.

6.6 Symbolic links

A symbolic link, or soft link, is a special file whose contents are another path name. When the kernel encounters a symlink during lookup, it reads the stored path and continues resolution using that path.

Definition

A **symbolic link** is a special file containing a path to another file-system object. Unlike a hard link, it has its own inode and can point to a path that may not currently exist.

Symbolic links are more flexible than hard links. They can often cross file-system boundaries and can point to directories. They can also become dangling: if the target path is removed, the symlink remains but no longer resolves to a valid object.

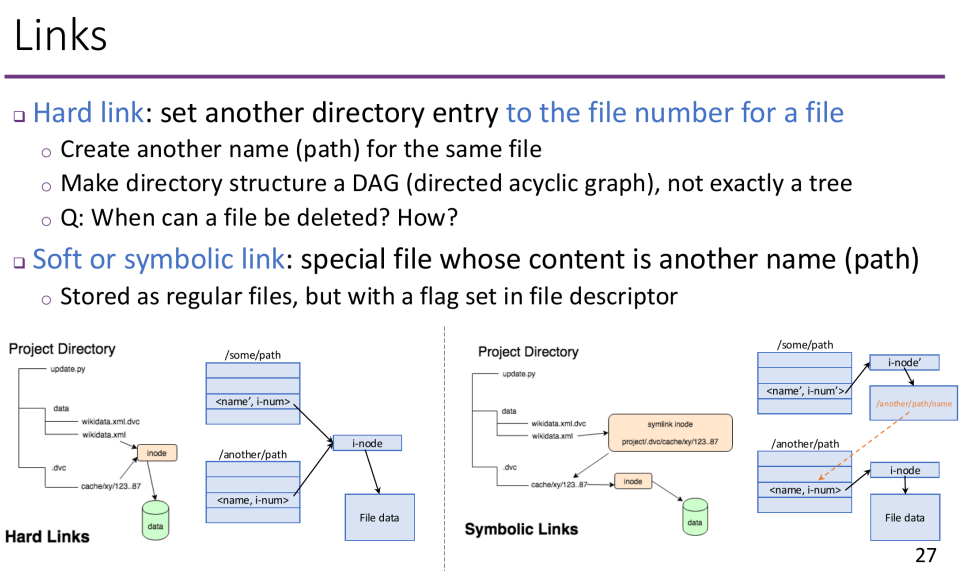


Figure 7: Hard links create multiple directory entries for the same inode. Symbolic links create a separate inode whose contents store another path. Hard links share file metadata and data directly; symbolic links redirect name resolution.

7 Buffer Cache and File-System Performance

7.1 Why cache disk blocks?

Storage is much slower than memory. Even SSDs have latencies far above ordinary DRAM access, and hard disks are slower still. The buffer cache uses part of main memory to hold

recently accessed disk blocks. If a later operation needs the same block, the system can avoid a physical storage access.

The buffer cache is implemented in operating-system software. It can cache many kinds of blocks:

- data blocks of regular files;
- directory data blocks;
- inode blocks;
- indirect blocks;
- free-space bitmaps and other allocation metadata.

The cache is usually managed at block granularity. Replacement policies are often related to least-recently-used (LRU), although production kernels use more sophisticated policies to handle streaming workloads, dirty pages, memory pressure, and metadata priority.

7.2 Read path with a buffer cache

A simplified read path looks like this:

1. The process calls `read(fd, buf, n)`.
2. The kernel translates `fd` to open-file state and an inode.
3. The kernel maps the current file offset to one or more logical file blocks.
4. For each logical block, the kernel finds the disk block number.
5. The buffer cache is checked.
6. On a hit, bytes are copied from cached memory to the user buffer.
7. On a miss, the block is read from storage into the cache, then copied.

The cache also benefits metadata lookup. Repeatedly resolving paths under the same directories is much faster if directory blocks and inodes are cached.

7.3 Buffer cache and virtual memory

Many operating systems unify the buffer cache and the virtual-memory page cache. This avoids double caching for memory-mapped files.

Consider a file page accessed through memory mapping. If the page is not resident, a page fault causes the kernel to fetch file data. If file data is already present in a separate buffer cache, the kernel may still copy it into a VM page, creating two memory copies of the same data. A unified cache makes the file-backed VM page itself the cached object. Then ordinary `read`/`write` and memory-mapped I/O can share the same physical page.

This unification matters because memory is valuable. Double caching wastes capacity and can reduce effective cache hit rate. It also complicates consistency: if one copy is updated through `write` and another through a memory map, the system must ensure both views agree.

7.4 Synchronous writes and delayed writes

When a cached block is modified, the system must decide when to write it back to storage.

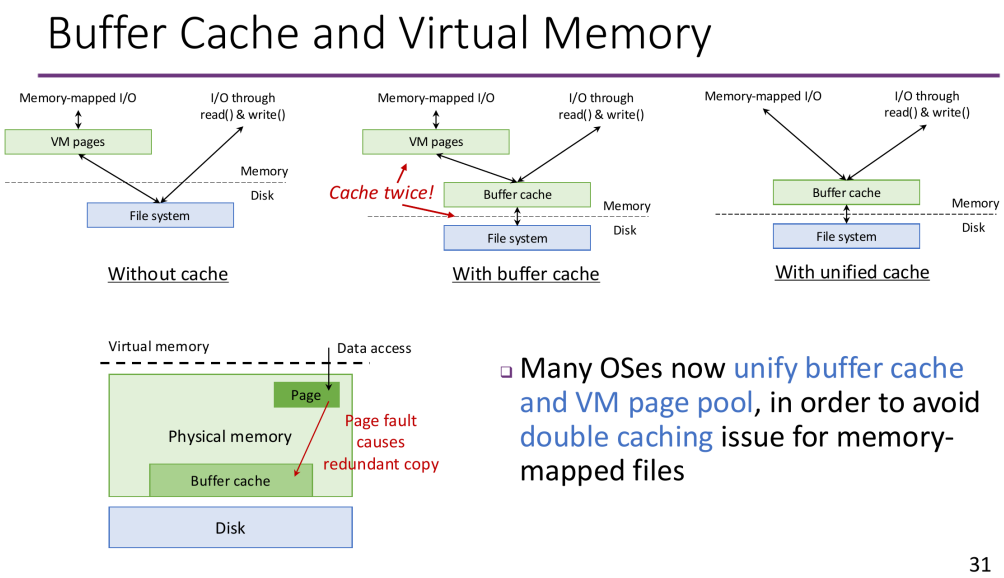


Figure 8: A separate buffer cache and virtual-memory page pool can cache the same file data twice. Memory-mapped I/O may fault a file page into physical memory, while read/write I/O may also keep the same block in the buffer cache. A unified cache avoids redundant copies and lets both access paths share physical pages.

7.4.1 Synchronous writes

With synchronous writes, the system writes through to storage immediately or before reporting completion. This is safer because a crash after completion is less likely to lose acknowledged data. It is slower because the caller waits for storage latency, and repeated writes to the same block may cause redundant disk traffic.

7.4.2 Delayed writes

With delayed writes, the system marks the cached block dirty and returns quickly. Later, the kernel flushes dirty blocks because of time, memory pressure, explicit synchronization, or file close. Delayed writes are fast and can combine many small updates into one storage write. They are also dangerous: a crash before flush can lose dirty data.

The Unix `sync` system call asks the system to schedule writeback of dirty blocks. The `fsync(fd)` call is stronger for a particular file: it asks that the file's dirty data, and metadata needed to retrieve it, reach stable storage. The details are subtle because disks and SSDs may also have internal volatile caches.

Deep Dive

A successful `write` is not the same as durable storage. For applications such as databases, editors, and package managers, the timing of `fsync`, `rename`, and metadata updates can determine whether a crash leaves old data, new data, or an inconsistent mix. Journaling and transactions are designed to control these failure cases.

7.5 Performance pitfalls in user programs

User programs can interact badly with file-system performance.

First, reading or writing one byte at a time using system calls is expensive because each call crosses the user-kernel boundary. Library buffering, such as C's `stdio`, batches small operations.

```c
/* Slow pattern: one system call per byte. */
char c;
while (read(fd, &c, 1) == 1) {
    process(c);
}

/* Better: read a block of bytes and process them in memory. */
char buf[8192];
ssize_t n;
while ((n = read(fd, buf, sizeof(buf))) > 0) {
    for (ssize_t i = 0; i < n; ++i) {
        process(buf[i]);
    }
}
```

Listing 3: Byte-at-a-time system calls are usually much slower than buffered I/O.

Second, frequent forced synchronization can dominate runtime. Calling `fsync` after every small write may be necessary for some correctness requirements, but it should be an explicit design choice. Third, random writes to many blocks can defeat locality and cause write amplification in storage devices.

8 Protection

8.1 Protection goals

Protection prevents accidental and malicious misuse of resources. The accidental case is just as important as the malicious one. A buggy program should not overwrite another user's files, corrupt kernel memory, or change system configuration. Isolation limits the damage caused by mistakes and attacks.

Protection has three conceptual parts:

1. **Authentication**: identify the responsible principal. Who are you?
2. **Authorization**: decide which operations the principal may perform on which objects. What can you do?
3. **Access enforcement**: ensure that every attempted operation obeys the authorization decision.

Definition

A **principal** is an entity to which responsibility and permissions are assigned. In an operating system, principals often include users, groups, processes, or services.

8.2 Authentication with passwords

Passwords are the traditional authentication mechanism. The system accepts a user identity if the user can demonstrate knowledge of the correct secret. Good passwords should be hard to guess, but the system side is equally important: how should the system store password information?

Storing plaintext passwords is dangerous. If an attacker obtains the password file, all accounts are compromised. The standard improvement is to store a one-way hash of the password rather than the password itself. During login, the system hashes the presented password and compares the result to the stored hash.

However, simple hashing is vulnerable to dictionary or rainbow-table attacks. An attacker can precompute hashes for many possible passwords and then look up stolen hashes. A salt defeats shared precomputation.

Definition

A **salt** is a random value stored with a password hash. The system stores $H(password \parallel salt)$ rather than $H(password)$. The salt need not be secret; its role is to force attackers to build a different dictionary for each salt.

With salts, two users with the same password have different stored hashes, and a precomputed table for one salt does not help against another. Modern password systems also use deliberately slow password-hashing functions to make guessing more expensive.

Other authentication methods include physical badges, USB security keys, biometrics, one-time passwords, and multi-factor authentication. The underlying principle remains the same: bind a system principal to evidence presented at login or request time.

8.3 Attaching principals to processes

After login, the operating system attaches the authenticated identity to processes. In Unix-like systems, a shell runs with the user's UID and GID. When the shell starts programs, child processes inherit the identity, typically through `fork` and `exec`. The kernel protects these identity fields from arbitrary user modification because later access checks rely on them.

This is why process isolation matters. If one process could directly overwrite another process's credentials or kernel memory, authorization would be meaningless. The kernel must maintain the integrity of the principal attached to each process.

There are controlled exceptions. Mechanisms such as privilege elevation, setuid programs, and administrative tools can change effective identities under strict rules. These mechanisms are powerful and therefore security-sensitive.

8.4 Authorization as an access matrix

Authorization asks which principals may perform which operations on which objects. For files, the objects are files and directories, and operations include read, write, execute, create, delete, and rename. Logically, authorization can be represented as an access matrix. Rows are principals; columns are objects; each entry records allowed operations.

	file A	**file B**	**directory C**
Alice	read, write	read	read, execute
Bob	read	-	execute
Carol	-	read, write	read, write, execute

The full matrix is usually too large and sparse. A system may have many users and millions of files, but most users care about only a small subset of files, and most files are accessible to only a few principals. The two classic compressions are access-control lists and capabilities.

8.5 Access-control lists

An access-control list stores authorization information with the object. It is a column-wise representation of the access matrix. For a file, the ACL says which users or groups may perform which operations.

Definition

An **access-control list** (ACL) is object-centered authorization metadata. It records which principals may perform which operations on a particular object.

Unix's classic permission model is a compact ACL: three classes of principals - owner, group, and others - each get three permission bits - read, write, and execute. Thus each file has nine basic permission bits:

$$(user, group, others) \times (read, write, execute).$$

For regular files, read means reading file contents, write means modifying contents, and execute means using the file as a program or script. For directories, the meanings are subtler. Read permits listing names in the directory. Write permits adding, removing, or renaming entries, usually in combination with execute. Execute on a directory permits traversing it during path lookup.

```
/* Conceptual examples, not C statements. */
chmod u+r file      /* owner may read */
chmod g-w file      /* group may not write */
chmod o+x script    /* others may execute */
chmod 755 program   /* rwx for owner, rx for group and others */
```

Listing 4: Unix permission bits can be changed through chmod-style operations.

ACLs are convenient when asking, "Who may access this object?" The information is close to the object and can be checked during object access. They are less convenient when asking, "What objects can this principal access?" Answering that question may require scanning many objects. ACLs can also become large if permissions cannot be summarized by groups.

8.6 Capabilities

A capability stores authorization information with the principal. It is a row-wise representation of the access matrix. A capability list says which objects the principal may access and how.

Definition

A **capability** is an unforgeable reference that grants a principal permission to access an object in specified ways.

Capabilities must be unforgeable: a process should not be able to invent a capability for an object it was not granted. They should also be revocable: the system should be able to remove access when necessary. Capabilities are often maintained inside the operating system or protected by cryptographic mechanisms.

Page tables are a useful example. A process can access only physical memory frames mapped in its page table. The mapping both names the object and grants permitted access modes such as read, write, or execute. If a physical frame is not mapped, the process cannot even address it. This combines naming with authorization.

File handles also have capability-like behavior. Once a process has a valid handle, it can perform operations permitted for that open file, even if later name lookup would be different. The handle is interpreted by the kernel and cannot normally be forged by choosing an arbitrary integer.

Capabilities are convenient when asking, "What can this principal access?" They are less convenient when asking, "Who can access this object?" Revocation can also be difficult if capabilities have been copied or delegated.

8.7 Access enforcement and the trusted computing base

Authentication and authorization rules are only useful if some component enforces them. This component has high privilege and must protect authentication data, authorization metadata, and the reference monitors that check access. Because it has great power, it should be as small and simple as possible.

Traditionally, the operating-system kernel is the enforcement point. The kernel mediates file-system operations, memory mappings, process creation, and device access. However, modern kernels are large and complex, which increases the risk of bugs and vulnerabilities.

One research direction is a security kernel: limit full privilege to a small part of the system and make other components manage only resources they are authorized to manage. Exokernel-style designs explored this idea by separating protection from higher-level resource abstractions. Security kernels and microkernel-inspired designs aim to reduce the trusted computing base, although mainstream operating systems still often place substantial trust in large kernels.

Deep Dive

Cloud computing raises an even harder question: what if the operating system itself is not fully trusted? If a program runs on someone else's machine, the user may trust the processor vendor or cryptographic protocol more than the cloud provider's OS. Trusted execution environments, hardware security mechanisms, and privacy-preserving computation are attempts to compute safely under weaker trust assumptions.

9 Worked Examples and Design Comparisons

9.1 Counting block accesses in path lookup

Suppose a cold cache, one-block directories, and Unix-like inodes. Resolving `/my/book/count` takes seven metadata reads before reading the file's data. If the file is then read sequentially and its first data block is not cached, an eighth read fetches that block. With a warm cache, repeated lookups may require no physical reads at all.

This example shows why directory and inode caching are important. Path lookup is metadata-intensive. A program that repeatedly opens many small files may spend more time resolving names and reading metadata than reading file contents.

9.2 Maximum file size with multi-level indexing

With 4 KiB blocks and 4-byte pointers, one indirect block holds 1024 pointers. A design with 12 direct pointers, one single-indirect pointer, and one double-indirect pointer can address

$$12 + 1024 + 1024^2$$

data blocks. Multiplying by 4096 bytes gives

$$(12 + 1024 + 1048576) \times 4096 \approx 4\ \text{GiB} + 4\ \text{MiB} + 48\ \text{KiB}.$$

The exact maximum is less important than the design lesson. A few metadata pointers in an inode can cover small files quickly while expanding to support large files through sparse, on-demand metadata blocks.

9.3 Allocation strategy comparison

Strategy	Strengths	Weaknesses	Best suited for
Contiguous allocation	Very simple; excellent sequential locality; constant-time random block calculation; tiny metadata.	External fragmentation; difficult growth; may require moving files.	Read-only media, fixed-size files, extent-like runs.
Linked allocation	No external fragmentation; easy append and insertion; high space utilization.	Poor random access; poor physical locality; pointer overhead.	Simple file systems, small removable media, FAT-like designs with table in memory.
Indexed allocation	Direct access to block pointers; no need for contiguous data; data blocks contain only data.	Index overhead; fixed index size can be too small or wasteful.	General-purpose file systems with multi-level or extent-based indexing.
Multi-level indexing	Fast small files; supports large files; allocates metadata on demand.	More complex; large files require extra metadata reads; worst-case random access may touch several metadata blocks.	Unix-like general-purpose file systems.

10 Summary

A file system is an abstraction layer that turns a linear block-addressed storage device into named, durable byte sequences organized by directories and protected by permissions. The storage hardware provides sectors, blocks, and LBAs; the operating system builds files, metadata, names, and access rules above that interface.

The central metadata structure is the inode or file control block. It persists on disk and is cached in kernel memory while the file is open. Opening a file resolves a path to a file number, locates the inode, creates open-file state, and returns a protected file handle to user space. Two-level open-file tables separate per-process handles from system-wide shared state.

File data can be organized by contiguous allocation, linked allocation, indexed allocation, or multi-level indexing. Contiguous allocation is fast but fragments and grows poorly. Linked allocation is flexible but weak for random access. Indexed allocation supports direct access without requiring contiguous blocks, and multi-level indexing adapts to both small and large files. Free space can be tracked by linked lists or bitmaps; locality-aware bitmap allocation benefits from keeping the disk from becoming completely full.

Directories map names to file numbers. Unix-like directories are special files whose data contains name-to-i-number mappings. Path resolution alternates between inodes and directory data blocks. Hard links create multiple names for the same inode; symbolic links store another path and redirect lookup. Current working directories and the special entries `.` and `..` make relative naming convenient.

The buffer cache uses memory to avoid repeated storage access and may be unified with the virtual-memory page cache to prevent double caching. Writes can be synchronous or delayed; delayed writes improve performance but require explicit synchronization or recovery mechanisms to control crash behavior.

Protection consists of authentication, authorization, and enforcement. Password systems should store salted hashes rather than plaintext. After authentication, the operating system attaches principals to processes. Authorization can be viewed as a sparse access matrix, compressed either by columns as ACLs or by rows as capabilities. The kernel or security kernel enforces access control, and the size of the trusted computing base directly affects system security.

11 Quizzes & Exercises

11.1 Conceptual quiz

1. Explain the distinction between a block and a sector in the context of storage devices.
2. What is the function of the Logical Block Address (LBA), and how does the hardware controller use it?
3. Contrast the user's view of a file with the operating system's internal view.
4. Describe the purpose and operation of the `/proc` file system found in Unix and Linux.
5. What is a file descriptor or inode, and what specific metadata does it typically contain?
6. Outline the steps the operating system performs during a file `open` operation.
7. Explain the three primary file access patterns: sequential, random, and keyed/indexed.
8. What are the primary advantages and disadvantages of contiguous data block allocation?
9. How does the Windows FAT system manage file links?
10. How does the 4.3 BSD Unix multi-level index handle different file sizes?

11.2 Answer key

1. A sector is the physical unit of storage on a device. A block is the logical transfer and allocation unit used by the operating system or file system. A block contains one or more sectors.
2. LBA provides a linear address space from 0 to the maximum logical block number. The disk or SSD controller translates LBAs to physical positions, hiding bad-sector remapping, wear-leveling, and other device-specific details.
3. Users see a file as a named, durable byte sequence. Internally, the operating system sees a file as metadata plus a collection of disk blocks that may not be physically contiguous.
4. `/proc` exposes system and process information through file-like operations. Reading returns system state; writing to selected entries can change configuration.
5. An inode is persistent file metadata. It typically stores file size, block locations, timestamps, ownership, group, permissions, type bits, and link count.
6. The OS resolves the path through directories, obtains a file number, locates the inode, brings necessary metadata into kernel memory, creates open-file table entries, and returns a per-process file handle.
7. Sequential access reads or writes bytes in order. Random access seeks directly to a byte offset. Keyed or indexed access searches based on content rather than offset and usually requires additional indexing structures.

8. Contiguous allocation is simple, fast for sequential access, and supports direct random access. Its disadvantages are external fragmentation and difficulty growing files.

9. FAT stores linked-allocation pointers in a separate table. Each table entry corresponds to a disk block and stores the next block number, an end-of-file marker, or a free-block marker.

10. The inode uses direct pointers for small files, then single-indirect and double-indirect blocks for larger files. Direct pointers make small files fast, while indirect levels support large files with modest extra metadata.

11.3 Essay questions

1. **The evolution of storage abstraction.** Discuss how the transition from physical sectors to Logical Block Addresses and finally to high-level file interfaces represents a series of abstractions that benefit both the operating system and the end user.

2. **Comparing block allocation strategies.** Critically analyze the trade-offs between contiguous, linked, and indexed data block organization, focusing on fragmentation, file growth, and random access performance.

3. **Metadata persistence and management.** Explain why the inode must exist both on disk and in kernel memory. Describe the consequences for performance and reliability if this metadata were managed incorrectly.

4. **Free space management and disk performance.** Evaluate the bitmap approach to free-space management used in BSD Unix. Discuss the significance of reserving capacity and its impact on the space-vs-time trade-off.

5. **Naming and directory structures.** Compare early single-directory systems with modern hierarchical structures. How does treating directories as files simplify operating-system design?

11.4 Additional practice problems

1. A file system has 4 KiB blocks. How many blocks are allocated to a 1-byte file, a 4096-byte file, and a 4097-byte file? For each case, compute internal fragmentation.

2. Suppose an indexed file uses one 4 KiB index block and 4-byte block pointers. What is the maximum file size if only that one index block is available?

3. In the BSD-style inode described in this lecture, determine whether byte offset 200 KiB is reached through a direct block, a single-indirect block, or a double-indirect block. Show your calculation.

4. A cold-cache lookup of `/a/b/c/d` assumes each directory fits in one block. How many metadata block reads are needed to resolve the path to the inode for `d`? Ignore symbolic links.

5. Explain why a hard-linked file cannot be deleted simply when one path name is removed.

6. Give an example of a dangling symbolic link. Why can symbolic links dangle while hard links generally cannot?

7. A program writes 100 bytes to a file and then exits without calling `fsync`. Explain why the data may or may not survive a sudden power loss.

8. Compare ACLs and capabilities using the access matrix. Which one makes it easier to answer "who can access this file," and which one makes it easier to answer "what files can this process access"?

9. Why is a file handle safer than returning a direct kernel pointer to an inode?

10. Explain how salts make rainbow-table attacks less practical even though salts are not secret.

11.5 Quick reference glossary

Term	Definition
Bitmap	Free-space representation using one bit per block, indicating whether each block is free or allocated.
Block	Logical transfer and allocation unit for secondary storage.
Capability	Unforgeable reference granting a principal access to an object in specified ways.
Directory	File-system object mapping names to file numbers.
External fragmentation	Free space exists in total but not in a sufficiently large contiguous region.
FAT	File Allocation Table; a separate table that stores next-block links for files.
File handle	Per-process integer returned by `open` and interpreted by the kernel.
Hard link	Directory entry mapping another name to an existing inode.
Inode	Persistent metadata structure describing a file.
Internal fragmentation	Wasted space inside an allocated block, often in the last block of a file.
LBA	Logical Block Address; a linear block number translated by the device controller to a physical location.
Symbolic link	Special file whose contents are a path to another object.
Salt	Random value combined with a password before hashing to prevent shared precomputed dictionaries.
Sector	Physical storage unit on a storage device.